WORKSHOP · 60 MIN

Agentic Coding

A Hands-On Workshop

Kassem Bagher

WHAT YOU'LL WALK AWAY WITH

Concrete deliverables, not vague understanding

01

A project memory file

CLAUDE.md / AGENTS.md generated yourself for a real repo.

02

One spec → generate → review cycle

Understand the loop end-to-end. Practice on a seeded repo after the session.

03

Five core principles

A mental model you can apply tomorrow on your own projects.

04

A practice repo

Yours to keep. Experiment after the session.

AGENDA · 60 MINUTES

Six parts. One core idea.

1	**Framing**	The one idea that holds it together	**5 min**
2	**Prompting essentials**	Four habits + bad-vs-good demo	**8 min**
3	**Context files: the project's memory**	CLAUDE.md, AGENTS.md, scoped rules	**12 min**
4	**Spec → generate → review**	The core loop, plus Superpowers	**17 min**
5	**Hands-on practice**	Run the loop on a seeded repo	**10 min**
6	**Q&A**	Anything goes	**8 min**

PART 01 · AUDIENCE CHECK

Quick show of hands

- **GitHub Copilot** — Raise your hand
- **Cursor** — Raise your hand
- **Claude (any version)** — Raise your hand
- **Never used an AI coding tool** — Raise your hand

Calibrates the rest of the talk. If "never used" wins, we slow down the prompting section.

THE ONE IDEA

The model has no memory between sessions.

Context is not a nice-to-have. It is the product. Everything in this workshop follows from that.

THE SPINE OF THE TALK

Five core principles

01	**No memory between sessions**	Context is the product. Add it or protect it.
02	**Spec before code**	What, why, constraints, what-not-to-do, references.
03	**CLAUDE.md (or AGENTS.md) is the shared brain**	Treat it like a README, for the AI.
04	**Review is not optional**	AI generates plausible-looking wrong code, confidently.
05	**Plans belong in git**	Chat dies. Commits live. Decisions get committed.

02

PART 02 · 8 MIN

Prompting essentials

Four habits that separate wasted time from useful output.

FOUR HABITS

DEMO #1 NEXT

How to prompt without wasting half your session

01

Be specific about what AND why

“Fix this bug” → “Function returns null on empty array; caller expects []. Don’t change the signature.”

02

Give one example

One concrete input/output beats three paragraphs of description. Works for code, tone, formatting.

03

State constraints and exclusions

“Don’t add new dependencies. Don’t touch auth.” What NOT to do is as valuable as what to do.

04

Iterate, don’t restart

Refine in place. A fresh chat throws away everything the model has learned this session.

03

PART 03 · 12 MIN

Context files

How the AI knows about your project. Every tool. One discipline.

THE PROBLEM

Every new session starts blank.

You re-explain the stack, the conventions, the gotchas, every time.

WITHOUT a context file

- Re-explain the project on every prompt
- AI guesses your stack, your style, your gotchas
- Same wrong answer, on a fresh day
- Tokens burned re-establishing context

WITH a context file

- Project memory loaded automatically
- Conventions and constraints up front
- AI starts informed, not from zero
- Write it once, every session benefits

PART 03

CONTEXT FILES BY TOOL

Same problem. Different filenames.

Tool	Project memory	Scoped rules
Claude Code	CLAUDE.md	.claude/rules/
Cursor	.cursor/rules/*.mdc · .cursorrules (legacy)	.cursor/rules/
GitHub Copilot	.github/copilot-instructions.md	.github/instructions/
Windsurf	.windsurfrules	.windsurf/rules/
Antigravity (Google)	GEMINI.md · AGENTS.md (native)	.agent/rules/
Aider	CONVENTIONS.md	-
AGENTS.md · cross-tool standard	**AGENTS.md**	**Linux Foundation · 60k+ repos**

The filename is cosmetic. The discipline is universal. Write the project's brain down once.

INSIDE A CLAUDE.MD / AGENTS.MD

DEMO #2 NEXT

A README, for the AI

- **Project overview**

  What this codebase is. What it does.

- **Stack**

  Language, frameworks, database, key libraries.

- **Conventions**

  Naming, structure, patterns. Be specific.

- **What NOT to do**

  Deprecated paths, gotchas, past mistakes.

- **Links to scoped rules**

  Root file stays short. Detail goes deeper.

RULE OF THUMB

The root context file should fit on one screen.

Scoped rule files hold the depth. Keep the entry point readable.

USING MULTIPLE AI TOOLS?

Don't duplicate your context file.

OPTION 1

Symlink

Mac / Linux. Fragile on Windows + mixed teams.

```
ln -s AGENTS.md CLAUDE.md
```

OPTION 2

Pointer file

Cross-platform. Most portable.

```
# CLAUDE.md
@AGENTS.md

# or just plain text:
See AGENTS.md for project context.
```

Only Claude Code needs the symlink. Cursor, Copilot, Antigravity, Windsurf, and Aider read AGENTS.md natively. Don't copy-paste; one file will go stale.

04

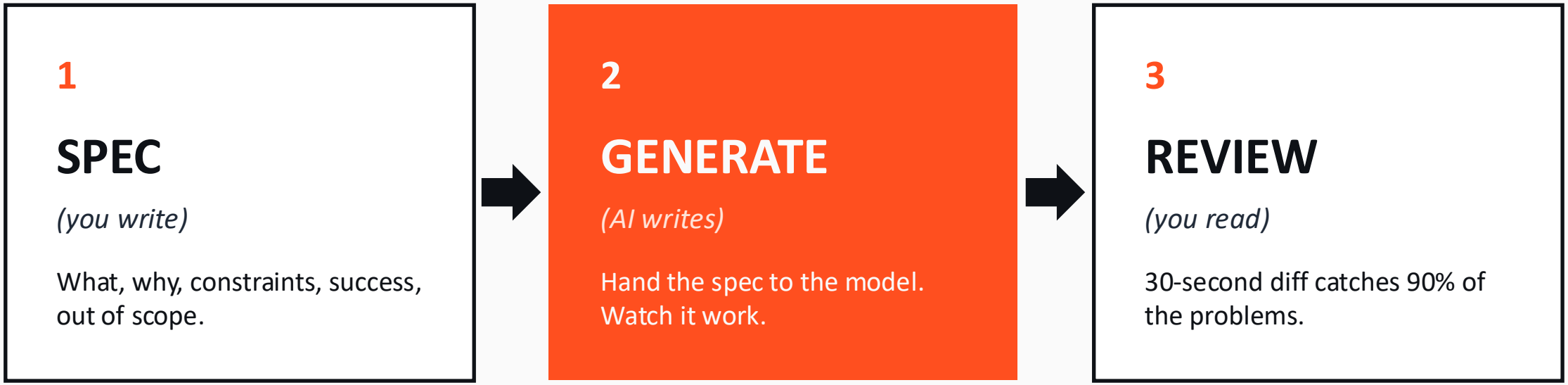
PART 04 · 17 MIN

Spec → Generate → Review

The core loop you'll run for every real task.

THE SPEC → GENERATE → REVIEW LOOP

Three steps. One feedback path.



↺ When the output is wrong, fix the SPEC and regenerate. Don't patch the code.

ANATOMY OF A SPEC

Five sections. Always.

```
## What
One sentence: what are we building?

## Why
One sentence: why does it need to exist?

## Constraints
- Don't do X
- Must preserve Y
- Follow the pattern in <file>

## Success criteria
- How do we know it worked?

## Out of scope
- What explicitly NOT to do
```

- **Reference existing files.**

  "Follow the pattern in userService.js" beats a paragraph of description.

- **"Out of scope" is the brake.**

  It stops the AI from rewriting half your project.

- **Success criteria = review checklist.**

  You already know what you're checking for.

WHY REVIEW IS NON-NEGOTIABLE

AI generates plausible-looking wrong code, confidently.

Hallucinated functions

Calls methods that don't exist on real objects.

Wrong conventions

Follows the conventions it thinks you want, not yours.

Quietly broken edge cases

Happy path works. Empty array, null, race condition: doesn't.

A 30-second diff review catches 90% of the problems.

PART 04

THE PRODUCTION VERSION

Superpowers

A free agentic skills framework that bakes the brainstorm → plan → execute → review loop into your coding agent automatically.

~175k
GitHub stars

14
skills shipped

Jan '26
in Claude Code marketplace

8+
hosts supported

github.com/obra/superpowers · MIT license · Free

HOW SUPERPOWERS WORKS

DEMO #3 NEXT

Skills that fire automatically

Phase	Skill	What it forces
Design	`brainstorming`	Asks clarifying questions before any code is written
Design	`writing-plans`	Breaks the spec into 2–5 min tasks with file paths
Setup	`using-git-worktrees`	Isolated worktree per task , parallel without clobbering
Execute	`subagent-driven-development`	Each task to a subagent with two-stage review
Execute	`TDD enforcement`	RED-GREEN-REFACTOR. Deletes code written before tests
Review	`requesting-code-review`	Runs between tasks. Blocks on critical issues
Debug	systematic-debugging	Root cause → pattern → hypothesis → implementation
Finish	`verification-before-completion`	Runs commands. Captures output as proof
Finish	finishing-a-development-branch	Merge, PR, keep, or discard , cleans up worktree

"Let's build X" → brainstorming fires before Claude writes a line. Your CLAUDE.md still wins.

SUPERPOWERS · INSTALL

One command. Restart. Done.

OFFICIAL MARKETPLACE · RECOMMENDED

```
/plugin install superpowers@claude-plugins-official
```

Stable, vetted in the Anthropic marketplace.

LATEST FEATURES

```
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace
```

Direct from the Superpowers repo. New skills land here first.

Restart Claude Code. Skills activate automatically based on what you say next.

HONEST CAVEAT

When NOT to use Superpowers

Quick one-off scripts

The brainstorming overhead is wasted. Write the throwaway and move on.

Exploring unfamiliar APIs

You want to poke around, not commit to a spec. Let curiosity drive.

Very small tweaks

A one-line fix doesn't need TDD or a code-review pass.

PART 05 · HANDS-ON · 10 MIN

Your turn: run the loop yourself

1	**Fork & clone the practice repo**	URL on screen. Use whatever editor you came with.
2	**Generate your project memory file**	CLAUDE.md, AGENTS.md, GEMINI.md, .cursor/rules. Your tool, your filename.
3	**Write one scoped rule file**	Pick a convention you spot in the codebase. Document it.
4	**Fix one seeded bug**	Pick from ISSUES.md. Spec → generate → review the diff.
5	**Add one small feature**	Same loop. See FEATURES.md for ideas.

Graded on the process, not the fix. A good spec with a visible review beats a working fix from “fix it.”

Q&A · PART 06

The discipline matters more than the tool.

Learn the loop, spec, generate, review, and you can move between Cursor, Claude, Copilot, or whatever comes next without losing your footing.

Questions?